Model Broadcast in TLA$^+$

DARE Project Presentations
31.10.25

Arne Stoll

University of Kaiserslautern Landau
stolla@rptu.de
github.com/https://github.com/stolla99

Background

Background

Approach

Background → Approach → Broadcast


Background
Approach
Broadcast
Implementation

Background → Approach → Broadcast → Implementation → Wrap up

Temporal Logic of Actions: TLA$^+$

- Specification language for distributed and asynchronous systems
  - Introduced by Lamport, based on temporal logic
- Built on first-order logic and set theory
- Captures system behavior via:
  - $\mathtt{Init}$ and $\Box[\mathtt{Next}]_{\langle \mathtt{vars} \rangle}$
  - Actions describing state transitions
  - Safety and liveness properties
- Operators for model checking
  - $\Diamond F$, $\Box G$, $\Diamond\langle A \rangle_v$, $F \leadsto G$, $\Diamond\Box F$, $\Box\Diamond F$

PlusCal: Algorithm Language for TLA$^+$

- Any PlusCal algorithm can be automatically translated into TLA$^+$

PlusCal: Algorithm Language for TLA$^+$

- Any PlusCal algorithm can be automatically translated into TLA$^+$
- Key features:
  - Rich mathematical expressions (set theory, logic)
  - Nondeterminism via `either`
  - Explicit atomic steps via labels (become TLA$^+$ actions)

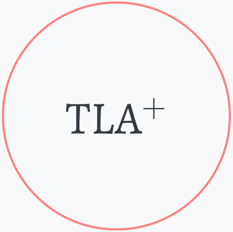
PlusCal: Algorithm Language for TLA$^+$

- Any PlusCal algorithm can be automatically translated into TLA$^+$
- Key features:
  - Rich mathematical expressions (set theory, logic)
  - Nondeterminism via `either`
  - Explicit atomic steps via labels (become TLA$^+$ actions)
- Language constructs:
  - Record access with `r.a`, nested datatypes
  - Extended control flow: `if`, `elsif`
  - Procedures with `call`, `return`, macros

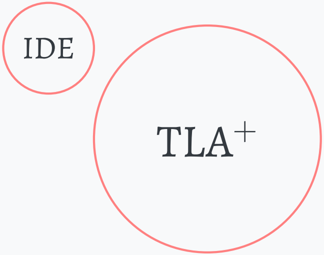
PlusCal: Algorithm Language for TLA$^+$

- Any PlusCal algorithm can be automatically translated into TLA$^+$
- Key features:
  - Rich mathematical expressions (set theory, logic)
  - Nondeterminism via `either`
  - Explicit atomic steps via labels (become TLA$^+$ actions)
- Language constructs:
  - Record access with `r.a`, nested datatypes
  - Extended control flow: `if`, `elsif`
  - Procedures with `call`, `return`, macros
- Labeling rules ensure correct translation:
  - Mandatory in `while`-loops and after `either`, `call`, `return`
  - Required between consecutive writes to the same variable

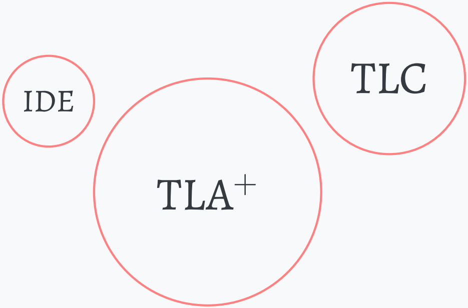
Landscape



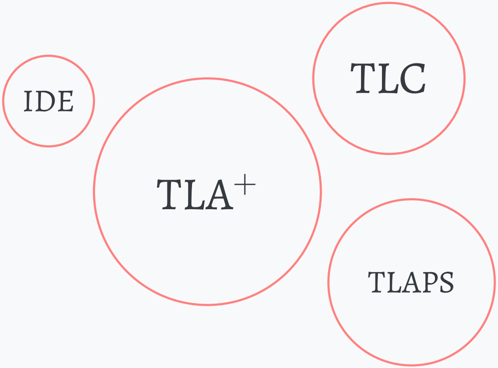
Landscape



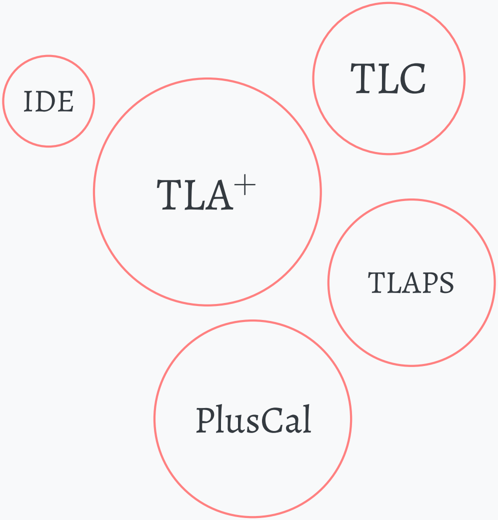
Landscape



Landscape



Landscape



Approach

PlusCal algorithm

Approach

PlusCal algorithm

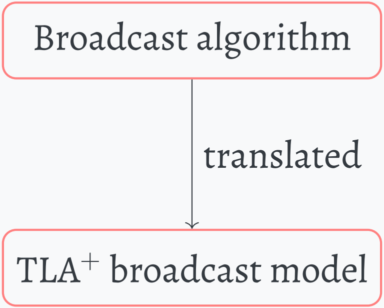
$\downarrow$ translated

TLA$^+$ spec

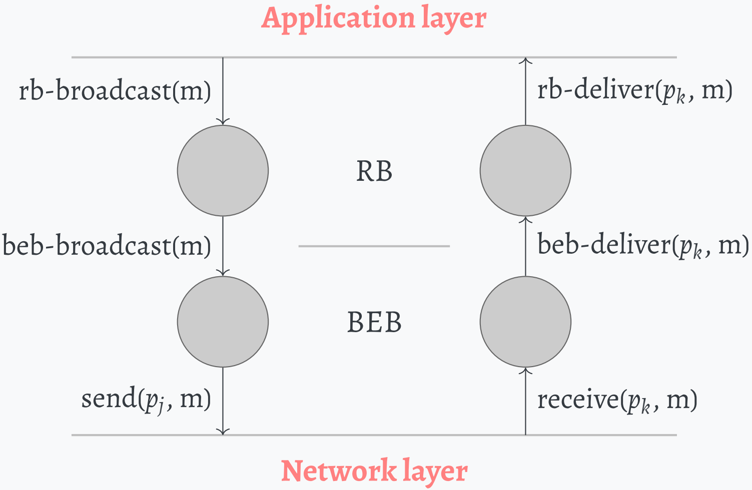
Approach

Broadcast algorithm

Approach



Broadcast recap



Broadcast can be done:

Broadcast can be done:

- Best effort

Broadcast can be done:

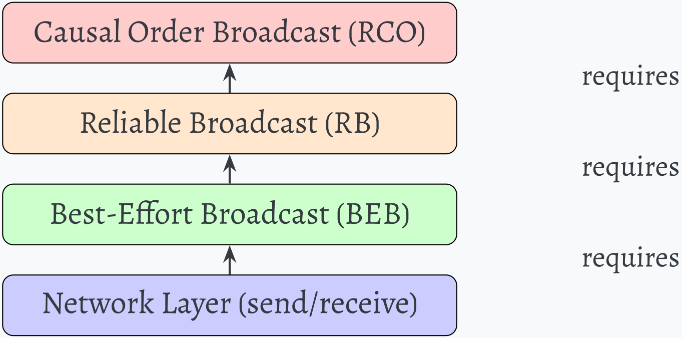
- Best effort
- Reliable

Broadcast can be done:

- Best effort
- Reliable
- Causal

Implementation Approach

Layered architecture: Each broadcast module builds upon the previous layer



Process Architecture

Two-process design: Separation of concerns between broadcasting processes and messaging (actor)

Broadcast Process(es)

- Implements broadcast algs
- Handles protocol logic

delegate send

Actor Process

- Sends messages
- Cannot broadcast

The broadcast process uses the actor process for actual network communication

Code demo for broadcast algorithm

...

A simple property

$$P_{EventualDelivery} = \Diamond \left(\bigwedge_{n \in Nodes} |delivered[n]| \neq 0 \right)$$

where

$Nodes = \{P_1, P_2, P_3\}$ and

$delivered = \{m_1, m_2, \ldots\}$

Conclusions

Pros

- Easy to use
- Fast to develop

Conclusions

Pros

- Easy to use
- Fast to develop

Cons

- Less expressive compared to TLA$^+$
- It is not like Erlang
  - message sending setup
- Harder defining properties
  - due to TLA$^+$ model

Thank you for your ***ATTENTION***